Практическое занятие №7. Структурирование Приложений FastAPI.

2. Структурирование Приложений FastAPI

В предыдущем материале были рассмотрены основные шаги, связанные с пониманием FastAPI и созданием приложения FastAPI. Приложение, которое мы создали до сих пор, однофайловое приложение todo, демонстрирующее гибкость и мощь FastAPI. Ключевым выводом из предыдущих работ является простота создания приложения с использованием FastAPI. Однако необходимо правильно структурировать приложение с повышенной сложностью ифункциональностью.

Структурирование относится к размещению компонентов приложения в организованном формате, который может быть модульным для улучшения читабельности кода и содержимого приложения. Приложение с правильной структурой обеспечивает более быструю разработку, более быструю отладку иобщее повышение производительности.

К концу работы вы будете владеть знаниями о том, что такое структурирование и как структурировать свой API. Здесь вы затронете следующие темы:

- Структурирование маршрутов и моделей приложений
- Реализация моделей для API планировщика

В этом материале мы создадим планировщик событий. Давайте разработаем структуру приложения, чтобы она выглядела так:

```
planner/
  main.py
  database/
    __init__.py
    connection.py
  routes/
    __init__.py
    events.py
    users.py
  models/
    __init__.py
    events.py
    users.py
```

Первый шаг — создать новую папку для приложения. Он будет называться планировщик:

```
$ mkdir planner && cd planner
```

Во вновь созданной папке планировщика, создайте файл ввода, `main.py`, и три подпапки– `database`, `routes`, and `models`:

```
$ touch main.py
$ mkdir database routes models
```

Затем создайте `__init__.py` в каждой папке:

```
$ touch {database,routes,models}/__init__.py
```

В папке `database`, давайте создадим пустой файл, `database.py`, который будет обрабатывать абстракции и конфигурации базы данных, которые мы будем использовать в следующей работе:

```
$ touch database/connection.py
```

В папках `routes` and `models`, мы создадим два файла: `events.py` и `users.py`:

```
$ touch {routes,models}/{events,users}.py
```

Каждый файл имеет свою функцию, как указано здесь:

- Модули в пакете `route`:
  - `events.py`: Этот модуль будет обрабатывать операции маршрутизации,такие как создание, обновление и удаление событий.
  - `users.py`: Этот модуль будет обрабатывать операции маршрутизации,такие как регистрация и вход пользователей.
- Модули в пакете `models`:
  - `events.py`: Этот модуль будет содержать определение модели для операций с событиями.
  - `users.py`: Этот модуль будет содержать определение модели дляпользовательских операций.

Теперь, когда мы успешно структурировали наш API и сгруппировали похожие по функциям файлы в компоненты, давайте приступим к реализации приложения в следующем

разделе.

Создание приложения для планирования мероприятий

В этом разделе мы будем создавать приложение планировщика событий. В этом приложении зарегистрированные пользователи смогут создавать, обновлять и удалять события. Созданные события можно просмотреть, перейдя на страницу события, автоматически созданную приложением.

Каждый зарегистрированный пользователь и событие будут иметь уникальный идентификатор. Это сделано для предотвращения конфликтов при управлении пользователями и событиями с одним и тем же идентификатором.

Чтобы начать разработку, давайте создадим виртуальную среду и активируемее в каталоге нашего проекта:

```
$ python3 -m venv venv
$ source venv/bin/activate
```

Далее давайте установим зависимости приложения:

```
(venv)$ pip install fastapi uvicorn "pydantic[email]"
```

Наконец, сохраните требования в `requirements.txt`:

```
(venv)$ pip freeze > requirements.txt
```

Теперь, когда мы успешно установили наши зависимости и настроили среду разработки, давайте реализуем следующие модели приложения.

Реализация моделей

Давайте посмотрим на шаги для реализации нашей модели:

1. Первым шагом в создании нашего приложения является определение моделей для события и пользователя. Модели описывают, как данные будут храниться, вводиться и представляться в нашем приложении. На следующейдиаграмме показано моделирование пользователя и события, а также их отношения:

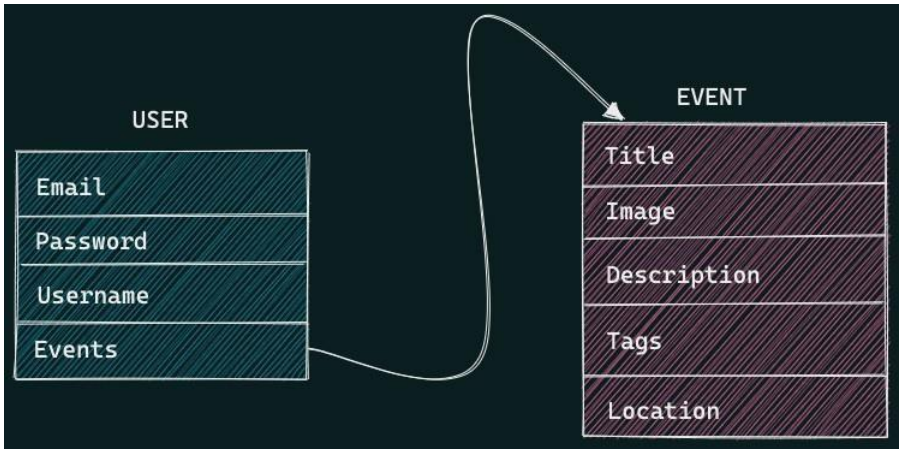


Рисунок 5.1 – Отношение моделей для пользователя и события

Как показано на предыдущей диаграмме модели, у каждого пользователя будет поле **Events**, представляющее собой список событий, на которые онимеет право собственности.

2. Определим модель Eventв models/events.py:

```
from pydantic import BaseModel
from typing import List


class Event(BaseModel):
    id: int
    title: str
    image: str
    description: str
    tags: List[str]
    location: str
```

3. Давайте определим подкласс `Config` в классе `Event`, чтобы показатьпример того, как будут выглядеть данные модели, когда мы посетим документацию:

```
    class Config:
        schema_extra = {
            "example"{
                "title": "FastAPI Book Launch",
                "image": "https:
                //linktomyimage.com/image.png",
                "description": "We will be discussing
                the contents of the FastAPI book in
                this event. Ensure to come with your
                own copy to win gifts!",
                "tags": ["python", "fastapi", "book",
                "launch"]
                "location": "Google Meet"
            }
        }
```

Наша модель событий в первом блоке кода содержит пять полей:

- Название события
- Ссылка на баннер изображения события
- Описание события
- Теги событий для группировки
- Место проведения

Во втором блоке кода мы определяем пример данных события. Это направлено на то, чтобы направлять нас при создании нового события изнашего API.

4. Теперь, когда мы определили нашу модель событий, давайте определим модель `User`:

```
from pydantic import BaseModel, EmailStr
from typing import Optional, List
from models.events import Event


class User(BaseModel):
    email: EmailStr
    password: str
    events: Optional[List[Event]]
```

Наша модель пользователя, определенная ранее, содержит следующие поля:

- Электронная почта пользователя
- Пароль пользователя
- Список событий, созданный пользователем, который по умолчанию пуст

5. Теперь, когда мы определили нашу модель `User`, давайте создадим пример, показывающий, как хранятся и устанавливаются пользовательскиеданные:

```
    class Config:
        schema_extra = {
            "example": {
                "email": fastapi@packt.com,
                "username": "strong!!!",
                "events": [],
           }
        }
```

6. Далее мы создадим новую модель, `NewUser`, которая наследуется от модели `User`; эта новая модель будет использоваться в качестве типа данныхпри регистрации нового пользователя. Модель `User` будет использоваться в качестве модели ответа, когда мы не хотим взаимодействовать с паролем, уменьшая объем работы, которую необходимо выполнить.
7. Наконец, давайте реализуем модель для входа пользователей в:

```
class UserSignIn(BaseModel):
    email: EmailStr
    password: str

    class Config:
        schema_extra = {
            "example": {
                "email": fastapi@packt.com,
                "password": "strong!!!",
                "events": [],
           }
        }
```

Теперь, когда мы успешно реализовали наши модели, давайте реализуем маршруты в следующем разделе.

Реализация маршрутов

Следующим шагом в создании нашего приложения является настройка системы

маршрутизации нашего API. Мы будем разрабатывать систему маршрутизации для событий и пользователей. Маршрут пользователя будет состоять из маршрутов входа, выхода и регистрации. Аутентифицированный пользователь будет иметь доступ к маршрутам для создания, обновления и удаления события, в то время как публика сможет просматривать событие после его создания. На следующей диаграмме показана взаимосвязь между обоими маршрутами:

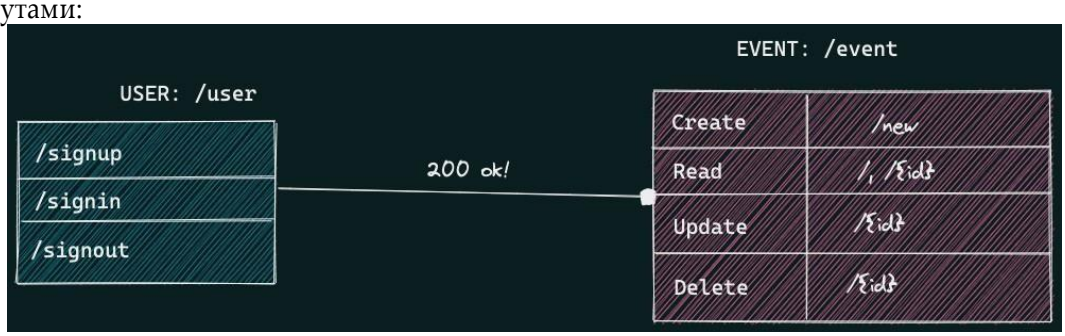


Рисунок 5.2 – Маршруты для операций пользователей и событий

Далее рассмотрим оба маршрута подробнее.

Маршруты пользователей

Теперь, когда у нас есть четкое представление о том, какие маршруты реализовать из *Рисунка 5.2*, мы начнем с определения пользовательских маршрутов в `users.py`. Давайте посмотрим на шаги:

1. Начните с определения основного маршрута регистрации:

```
from fastapi import APIRouter, HTTPException, status
from models.user import User, UserSignIn

user_router = APIRouter(
    tags=["User"]
)
users = {}
@user_router.post("/signup")
async def sign_new_user(data: NewUser) -> dict:
    if data.email in users:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="User with supplied username
            exists"
        )
    users[data.email] = data
    return {
        "message": "User successfully registered!"
    }
```

В маршруте регистрации, определенном ранее, мы используем базу данных в

приложении.

Маршрут проверяет, существует ли пользователь с похожим адресом электронной почты в базе данных перед добавлением нового.

2. Давайте реализуем маршрут входа:

```
@user_router.post("/signin")
async def sign_user_in(user: UserSignIn) -> dict:
    if users[user.email] not in users:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND
            detail="User does not exist"
        )
    if users[user.email].password != user.password:
            raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN
            detail="Wrong credentials passed"
        )

    return {
        "message": "User signed in successfully"
    }
```

В этом маршруте первым делом проверяется, существует ли такой пользователь в базе данных, и, если такой пользователь не существует, возникает исключение. Если пользователь существует, приложение проверяет, совпадают ли пароли, прежде чем вернуть успешное сообщение или исключение.

В наших маршрутах мы храним пароли в чистом виде без какого-либо шифрования. Это используется только в демонстрационных целях и является неправильной практикой в разработке программного обеспечения в целом.

3. Теперь, когда мы определили маршруты для пользовательских операций, давайте зарегистрируем их в `main.py` и запустим наше приложение. Давайте начнем с импорта наших библиотек и определения пользовательских маршрутов:

```
from fastapi import FastAPI
from routes.user import user_router

import uvicorn
```

4. Далее создадим экземпляр `FastAPI` и зарегистрируем маршрут и приложение:

```
app = FastAPI()

# Register routes
app.include_router(user_router, prefix="/user")
if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8080,
    reload=True)
```

В этом блоке кода мы создали экземпляр `FastAPI` и зарегистрировалимаршрут.

5. Затем мы используем метод `uvicorn.run()` для запуска нашего приложенияна порту `8080` и устанавливаем для перезагрузки значение `True`. В терминалезапустите приложение:

```
(venv)$ python main.py
INFO:     Will watch for changes in these directories:
['/Users/youngestdev/Work/Building-Web-APIs-with-FastAPI-
and-Python/ch05/planner']
INFO:     Uvicorn running on http://0.0.0.0:8080 (Press
CTRL+C to quit)
INFO:     Started reloader process [6547] using
statreload
```

```
INFO:     Started server process [6550]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

6. Теперь, когда наше приложение успешно запустилось, давайте проверим реализованные нами пользовательские маршруты. Начнемс регистрации пользователя:

```
(venv)$ curl -X 'POST' \
  'http://0.0.0.0:8080/user/signup' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "email": "fastapi@packt.com",
  "password": "Stro0ng!",
  "username": "FastPackt"
}'
```

Предыдущий запрос возвращает ответ:

```
{
  "message": "User successfully registered!"
}
```

7. Предыдущий ответ указывает на успешность выполненной операции. Давайте проверим маршрут входа:

```
(venv)$ curl -X 'POST' \
  'http://0.0.0.0:8080/user/signin' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "email": "fastapi@packt.com",
  "password": "Stro0ng!"
}'
```

Предыдущий ответ на запрос выглядит следующим образом:

```
{
  "message": "User signed in successfully"
}
```

8. Если мы передадим неверный пароль, наше приложение должно вернуть другое сообщение:

```
(venv)$ curl -X 'POST' \
  'http://0.0.0.0:8080/user/signin' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "email": "fastapi@packt.com",
  "password": "password!"
}'
```

Ответ на предыдущий запрос выглядит следующим образом:

```
{
  "detail": "Wrong credential passed"
}
```

Мы также можем просмотреть наши маршруты из интерактивной документации, предоставленной FastAPI, работающей на Swagger. Давайте посетим `http://0.0.0.0:8080/docs` в нашем браузере, чтобы получить доступ к интерактивной документации:

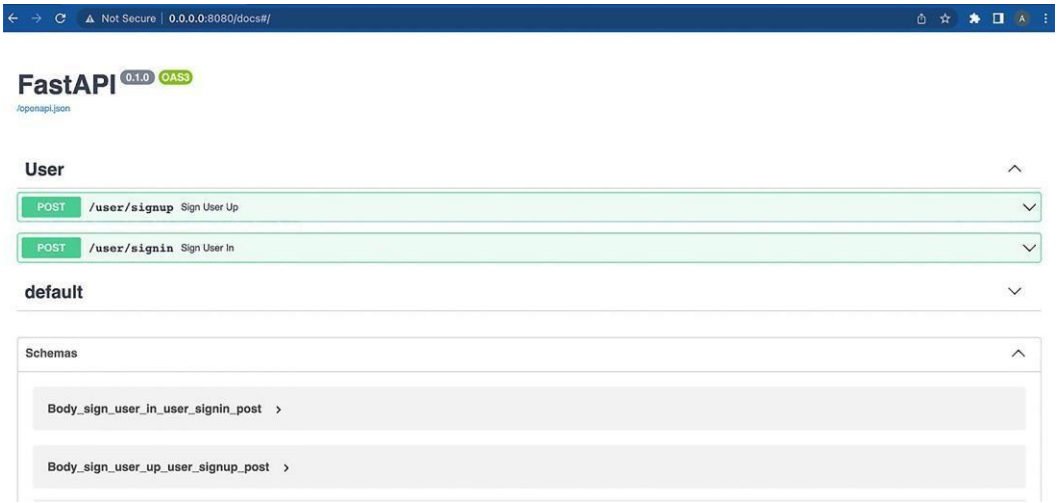


Рисунок 5.3 – Приложение планировщика, просмотренное на странице интерактивнойдокументации

Теперь, когда мы успешно реализовали пользовательские маршруты, давайте реализуем маршруты для событийных операций в следующем разделе.

Маршруты событий

После создания пользовательских маршрутов следующим шагом будет реализация маршрутов для событийных операций. Давайте посмотрим нашаги:

1. Начните с импорта зависимостей и определения маршрутизатора событий:

```
from fastapi import APIRouter, Body, HTTPException,
status
from models.events import Event
from typing import List

event_router = APIRouter(
    tags=["Events"]
)

events = []
```

2. Следующим шагом является определение маршрута для получения всех событий и события, соответствующего предоставленному идентификатору вбазе данных:

```
@event_router.get("/", response_model=List[Event])
async def retrieve_all_events() -> List[Event]:
    return events

@event_router.get("/{id}", response_model=Event)
async def retrieve_event(id: int) -> Event:
    for event in events:
        if event.id == id:
            return event
    raise HTTPException(
        status_code=status. HTTP_404_NOT_FOUND,
        detail="Event with supplied ID does not exist"
    )
```

Во втором маршруте мы вызываем исключение `HTTP_404_NOT_FOUND`, когда событие с предоставленным идентификатором не существует.

3. Давайте реализуем маршруты для создания события, удаления одногособытия и удаления всех событий, содержащихся в базе данных:

```
@event_router.post("/new")
async def create_event(body: Event = Body(…)) -> dict:
    events.append(body)
    return {
        "message": "Event created successfully"
    }

@event_router.delete("/{id}")
async def delete_event(id: int) -> dict:
    for event in events:
        if event.id == id:
            events.remove(event)
            return { "message": "Event deleted
            successfully" }
    raise HTTPException(
        status_code=status. HTTP_404_NOT_FOUND,
        detail="Event with supplied ID does not exist"
    )
@event_router.delete("/)
async def delete_all_events() -> dict:
    events.clear()
    return {
        "message": "Events deleted successfully"
    }
```

Мы успешно реализовали маршруты для событий. Маршрут UPDATE нужен для переноса приложения с использованием реальной базы данных.

4. Теперь, когда мы реализовали маршруты, давайте обновим нашу конфигурацию маршрута, чтобы включить маршрут события в `main.py`:

```
from fastapi import FastAPI
from routes.user import user_router
from routes.events import event_router
import uvicornapp = FastAPI()

# Register routes

app.include_router(user_router, prefix="/user")
app.include_router(event_router, prefix="/event")

if__name__== "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8080,
    reload=True)
```

Приложение автоматически перезагружается при каждом изменении. Проверим маршруты:

- Маршрут `GET` — следующая операция возвращает пустой массив, сообщаянам об отсутствии данных:

```
(venv)$ curl -X 'GET' \
  'http://0.0.0.0:8080/event/' \
  -H 'accept: application/json'
[]
```

Далее добавим данные в наш массив.

- Маршрут `POST` – в терминале выполните следующую команду:

```
(venv)$ curl -X 'POST' \
  'http://0.0.0.0:8080/event/new' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 1,
  "title": "FastAPI Book Launch",
  "image": "https://linktomyimage.com/image.png",
  "description": "We will be discussing the contents
  of the FastAPI book in this event.Ensure to come
  with your own copy to win gifts!",
  "tags": [
    "python",
    "fastapi",
    "book",
    "launch"
  ],
  "location": "Google Meet"
}'
```

Вот ответ:

```
{
  "message": "Event created successfully"
}
```

Эта операция прошла успешно, судя по полученному ответу. Теперь давайте попробуем получить конкретное событие, которое мы только что создали:

- Маршрут `GET`:

```
(venv)$ curl -X 'GET' \
  'http://0.0.0.0:8080/event/1' \
  -H 'accept: application/json'
```

Вот ответ:

```
{
  "id": 1,
  "title": "FastAPI BookLaunch",
  "image": "https://linktomyimage.com/image.png",
  "description": "We will be discussing the contents
  of the FastAPI book in this event.Ensure to come
  with your own copy to win gifts!",
  "tags": [
    "python",
    "fastapi",
    "book",
    "launch"
  ],
  "location": "Google Meet"
}
```

Наконец, давайте удалим событие, чтобы убедиться, что маршрут события работает:

- Маршрут `DELETE` – в терминале выполните следующую команду:

```
(venv)$ curl -X 'DELETE' \
  'http://0.0.0.0:8080/event/1' \
  -H 'accept: application/json'
```

5.
6.

Вот ответ:

```
{
  "message": "Event deleted successfully"
}
```

Если я повторю ту же команду, я получу следующий ответ:

```
(venv)$ {
  "detail": "Event with supplied ID does not exist"
}
```

Мы успешно реализовали маршруты и модели для нашего приложения-планировщика. Мы также протестировали их, чтобы оценить их рабочеесостояние.

Задание

1. Изучить как структурировать приложение FastAPI
2. реализовать маршруты и модели для приложения планирования событий.
3. Переформулировать диаграмму use case для текущего варианта приложения. Написать основные сценарии для вариантов использования.